Pipeline Completo de IA (End-to-End)

Django REST, MLflow, Docker, Magalu Cloud & Algoritmos Híbridos

Este tutorial cobre **todos** os passos necessários para criar um sistema de recomendação híbrido (Árvore de Decisão + KNN) e colocá-lo em produção.

---

1. Configuração e Instalação

O primeiro passo é preparar o terreno. Vamos criar o ambiente virtual.

```
# 1. Crie a pasta do projeto
mkdir projeto_magalu_ia
cd projeto_magalu_ia

# 2. Crie o ambiente virtual (venv)
python -m venv venv

# 3. Ative o ambiente (Windows)
venv\Scripts\activate

# 4. Instale a Stack completa
pip install django djangorestframework mlflow scikit-learn pandas numpy gunicorn
```

---

2. Machine Learning com MLflow

Treinamento híbrido: **Árvore de Decisão** (Formato) + **KNN** (Conteúdo Colaborativo).

Arquivo: `train.py`

```
import mlflow
import mlflow.sklearn
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
import pickle
import numpy as np

mlflow.set_experiment("Sistema_Hibrido_Magalu")
```

```
def train_models():
    with mlflow.start_run():
        # Dados: [Visual, Auditivo, Prático, Nota_Py, Nota_Design, Nota_Agile]
        X = np.array([
            [1,0,0, 5,1,1], [0,1,0, 1,5,2],
            [0,0,1, 2,2,5], [1,1,0, 4,1,2]
        ])

        # Targets: Formato e Tópico
        y_format = np.array([0, 1, 2, 0])
        y_topic = np.array([0, 1, 2, 0])

        tree = DecisionTreeClassifier(max_depth=3).fit(X[:, :3], y_format)
        knn = KNeighborsClassifier(n_neighbors=1).fit(X[:, 3:], y_topic)

        final_model = {"tree": tree, "knn": knn}

        with open("ia_models.pkl", "wb") as f:
            pickle.dump(final_model, f)

        print("Modelos salvos em ia_models.pkl")

if __name__ == "__main__":
    train_models()
```

---

3. Inicialização do Django REST

```
django-admin startproject core .
python manage.py startapp app_ia
```

Lógica da API (app_ia/views.py)

```
from rest_framework.views import APIView
from rest_framework.response import Response
from django.shortcuts import render
import pickle
import numpy as np
import os
```

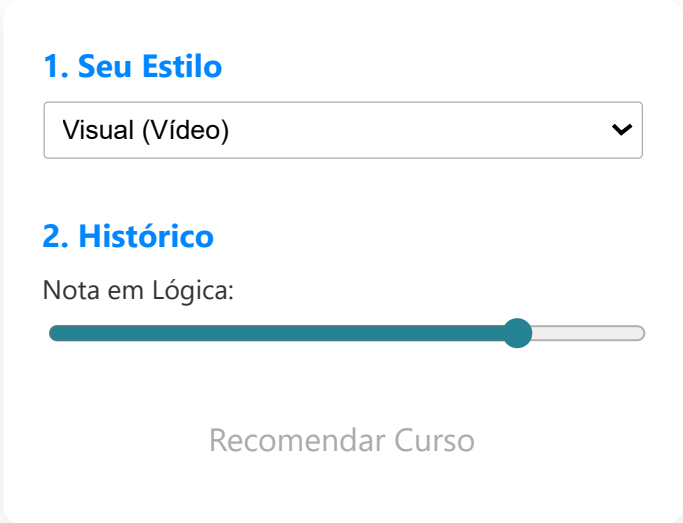
```
# Carregar modelo (Singleton)
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
with open(os.path.join(BASE_DIR, 'ia_models.pkl'), 'rb') as f:
    AI_MODELS = pickle.load(f)

def index(request): return render(request, 'index.html')

class PredictAPI(APIView):
    def post(self, request):
        style = request.data.get('style')
        grades = request.data.get('grades')

        fmt_id = AI_MODELS['tree'].predict([np.array(style)])[0]
        formats = {0: "Vídeo Aulas", 1: "Podcast", 2: "Bootcamp"}

        topic_id = AI_MODELS['knn'].predict([np.array(grades)])[0]
        topics = {0: "Python Avançado", 1: "UX/UI Design", 2: "Gestão Ágil"}

        return Response({"format": formats.get(fmt_id), "topic":
topics.get(topic_id)})
```

---

4. Frontend e Interface Gráfica

Arquivo: `app_ia/templates/index.html`

Magalu Education AI



Resultado: **Vídeo Aulas** de **Python Avançado**

```
<!-- Trecho do HTML -->
<button onclick="getAI()">🔍 Analisar com IA</button>
<div id="result"></div>

<script>
async function getAI() {
    // Lógica de fetch para /api/predict/
    // Exibe o resultado na div #result
}
</script>
```

---

5. Dockerização

Arquivo: `Dockerfile`

```
FROM python:3.10-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE 1
ENV PYTHONUNBUFFERED 1
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
```

```
EXPOSE 8000
CMD ["gunicorn", "core.wsgi:application", "--bind", "0.0.0.0:8000"]
```

---

6. Deploy na Magalu Cloud

> **Resumo:** Build da imagem -> Push para Registry -> Pull na Instância Magalu.

```
# 1. Build e Push
docker build -t app-magalu-ia .
docker push seu_usuario/app-magalu-ia:v1

# 2. Na Instância Magalu (SSH)
sudo docker run -d -p 80:8000 --restart always seu_usuario/app-magalu-ia:v1
```

Tutorial desenvolvido para implementação de IA na Magalu Cloud.